

Function Expressions - Simple Hinglish Summary

Source: <https://javascript.info/function-expressions>

Main baat

JavaScript me function koi "magical structure" nahi hai, balki ek **special type ki value** hai.

Pehle jo syntax use kiya wo **Function Declaration** hai:

```
javascript

function sayHi() {
  alert( "Hello" );
}
```

Function banane ka ek aur tarika hai: **Function Expression**. Isse kisi bhi expression ke beech me function bana sakte ho.

```
javascript

let sayHi = function() {
  alert( "Hello" );
};
```

- Yahan function `(=)` ke **right side** par bana hai, isliye ye Function Expression hai.
- `function` keyword ke baad **naam nahi** hai. Function Expression me naam chhodna allowed hai.
- Matlab: "function banao aur `sayHi` variable me rakh do."

(Aage aisi situations aayengi jahan function bana kar turant call ho jata hai ya kahin store nahi hota, tab wo anonymous rehta hai.)

1. Function ek value hai

Function kaise bhi bana ho, wo **ek value** hai. Hum use `alert` se print bhi kar sakte hain:

```
javascript

function sayHi() {
  alert( "Hello" );
}

alert( sayHi ); // function ka code dikhata hai
```

Dhyan: Last line function ko **run nahi karti**, kyunki `sayHi` ke baad **brackets `()` nahi hain**. Kuch languages me function ka naam likhte hi wo chal jata hai, JS me aisa nahi hai.

Function ko **dusre variable me copy** bhi kar sakte ho:

```
javascript

function sayHi() {    // (1) banao
  alert( "Hello" );
}

let func = sayHi;    // (2) copy karo

func();    // Hello    (3) copy chalao
sayHi();    // Hello    ye bhi chalta hai
```

Detail me:

1. Function Declaration `sayHi` variable me function banata hai.
2. Line (2) use `func` me copy karti hai. **Brackets nahi hain**. Agar `sayHi()` likhte to function ka **result** `func` me jata, function khud nahi.
3. Ab function `sayHi()` aur `func()` dono se call ho sakta hai.

Function Expression se `sayHi` banate to bhi sab same chalta.

Semicolon `;` kyu?

```
javascript

function sayHi() {
  // ...
}

let sayHi = function() {
  // ...
};
```

Function Expression `let sayHi = ...;` **assignment statement ke andar** bana hai, aur statement ke end me `;` lagana recommended hai. Ye function syntax ka hissa nahi hai. Ye wahi hai jo `let sayHi = 5;` me lagta hai.

2. Callback functions

Ab dekhte hain functions ko **values ki tarah pass** karna. Ek function `ask(question, yes, no)` likhte hain:

- `question`: sawal ka text
- `yes`: "Yes" par chalne wala function
- `no`: "No" par chalne wala function

javascript

```
function ask(question, yes, no) {
  if (confirm(question)) yes()
  else no();
}

function showOk() {
  alert( "You agreed." );
}

function showCancel() {
  alert( "You canceled the execution." );
}

// showOk aur showCancel ko arguments ki tarah pass kiya
ask("Do you agree?", showOk, showCancel);
```

`showOk` aur `showCancel` ko *callback functions* (ya sirf *callbacks*) kehte hain.

Idea ye hai: hum ek function pass karte hain aur ummeed karte hain ki zarurat padne par wo baad me "wapas call" hoga.

Function Expression se chhota code

javascript

```
function ask(question, yes, no) {
  if (confirm(question)) yes()
  else no();
}

ask(
  "Do you agree?",
  function() { alert("You agreed."); },
  function() { alert("You canceled the execution."); }
);
```

Yahan functions `ask(...)` call ke **andar hi** bane hain. Inka **naam nahi** hai, isliye inhe **anonymous** kehte hain. Ye `ask` ke bahar accessible nahi hain (kyunki kisi variable me

assign nahi hue), aur yahan hume yahi chahiye. Ye JavaScript ke **spirit** ke hisaab se bahut natural code hai.

Function ek "action" ko represent karne wali value hai. Normal values (string, number) **data** hote hain, aur function ko **action** samajh sakte ho. Hum use variables ke beech pass kar sakte hain aur jab chahein tab chala sakte hain.

3. Function Expression vs Function Declaration

Syntax ka fark

Function Declaration: function ek **alag statement** ki tarah main code flow me:

```
javascript

function sum(a, b) {
  return a + b;
}
```

Function Expression: function kisi **expression ke andar** bana hota hai. Yahan (=) ke right side par:

```
javascript

let sum = function(a, b) {
  return a + b;
};
```

Fark 1: Kab bante hain (sabse important)

Function Expression tab banta hai jab **execution us line tak pahunche**, aur usi waqt se use ho sakta hai.

Function Declaration **define hone se pehle bhi call ho sakta hai**. Global Function Declaration **poori script me** dikhta hai, chahe wo kahin bhi likha ho. Wajah: JS script chalane se pehle ek "initialization stage" me saare global Function Declarations dhundhkar bana leta hai, phir code chalata hai.

Ye chalega (Declaration):

```
javascript

sayHi("John"); // Hello, John

function sayHi(name) {
  alert( `Hello, ${name}` );
}
```

Ye nahi chalega (Expression):

```
javascript

sayHi("John"); // error!

let sayHi = function(name) { // (*) yahan tak pahunchne par hi banega
  alert( `Hello, ${name}` );
};
```

Fark 2: Block scope

Strict mode me, code block ke andar Function Declaration us block me har jagah dikhta hai, lekin block ke bahar nahi.

Maan lo `age` ke hisaab se `welcome()` function banana hai aur baad me use karna hai. Declaration se ye nahi chalega:

```
javascript

let age = prompt("What is your age?", 18);

if (age < 18) {

  function welcome() {
    alert("Hello!");
  }

} else {

  function welcome() {
    alert("Greetings!");
  }

}

welcome(); // Error: welcome is not defined
```

Kyunki Function Declaration sirf **apne block ke andar** dikhta hai.

Sahi tarika: Function Expression use karo aur use ek aise variable me assign karo jo `if` ke **bahar** declare ho.

```
javascript
```

```

let age = prompt("What is your age?", 18);

let welcome;

if (age < 18) {

    welcome = function() {
        alert("Hello!");
    };

} else {

    welcome = function() {
        alert("Greetings!");
    };

}

welcome(); // ab chalega

```

? operator se aur chhota:

```

javascript

let age = prompt("What is your age?", 18);

let welcome = (age < 18) ?
    function() { alert("Hello!"); } :
    function() { alert("Greetings!"); };

welcome(); // chalega

```

Kab kya chunein?

Rule of thumb: Function banana ho to **pehle Function Declaration** socho. Isse code organize karne ki zyada azaadi milti hai (function declare hone se pehle bhi call kar sakte ho). Ye **zyada readable** bhi hai, kyunki `function f(...) {...}` code me dhundhna `let f = function(...) {...};` se aasan hai.

Lekin agar Declaration kisi wajah se fit na ho, ya **conditional declaration** chahiye (jaise upar dekha), to **Function Expression** use karo.

Summary

- **Functions values hain.** Unhe assign, copy ya code me kahin bhi declare kiya ja sakta hai.
- Function agar **alag statement** ki tarah main code flow me hai, to wo **Function Declaration** hai.
- Function agar **expression ka hissa** hai, to wo **Function Expression** hai.
- **Function Declarations** code block chalne se **pehle** process hote hain. Wo block me har jagah dikhte hain.
- **Function Expressions** tab bante hain jab execution flow unhe **pahunche**.
- Zyadatar cases me **Function Declaration** behtar hai, kyunki wo declare hone se pehle bhi dikhta hai. Function Expression tab use karo jab Declaration kaam ke liye fit na ho.

Quick Comparison

Baat	Function Declaration	Function Expression
Syntax	<pre>function sum(a, b) { ... };</pre>	<pre>let sum = function(a, b) { ... };</pre>
Semicolon (;)	Nahi	Haan (assignment statement ka hissa)
Kab banta hai	Script shuru hone se pehle	Jab execution us line tak pahunche
Define hone se pehle call	Ho sakta hai	Nahi ho sakta
Block ke bahar dikhta hai?	Nahi (strict mode me)	Variable ke scope ke hisaab se
Kab use karein	Default choice	Conditional creation, callbacks

Yaad rakhne wali baatein

- `sayHi` (bina brackets) = function **value**, `sayHi()` = function **call**
- **Callback** = wo function jo dusre function ko argument ki tarah pass kiya jata hai
- **Anonymous function** = jiska naam nahi hota
- Declaration pehle, Expression tab jab zarurat ho